

Appendix

Video Files:

1) Single cylinder:

 CylinderVideoMAE423.mp4

2) Plane with rectangular wings:

 RectangleVideoMAE423.mp4

3) Plane with elliptical wings:

 EllipsoidVideoMAE423.mp4

4) Plane with tapered wings:

 TaperedVideoMAE423.mp4

Code:

1) Single cylinder:

```
% MAE 423 Final Project
%
% By Dini Kathriarachchi & Allison Lesser
% set constants for problem
n = 500; % vertical length
m = 200; % horizontal
length threshold_error =
0.05; F_overrelax = 1.5;
time_intervals = 10000;
numFrame = 0;
frame_store_rate = 10; %
how many frames are stored
% constants for air
U_inf = 5; % m/s
T_inf = 300; % K
therm_diffusivity = 2.2 * 10^-5; %
m^2/s kin_viscosity = 1.5 * 10^-5; Re =
200; % no units
delta_h = Re * kin_viscosity / (50*U_inf); % delta_t =
delta_h/(10*U_inf); % create cell to store frames psicell
= cell(ceil(time_intervals/frame_store_rate), 1);
omegacell = cell(ceil(time_intervals/frame_store_rate),
1); tempcell =
cell(ceil(time_intervals/frame_store_rate), 1);
% create grids of desired geometry (n x
m) psiGrid = zeros(n, m); omegaGrid =
zeros(n, m); uGrid = zeros(n, m); vGrid =
zeros(n, m); tempGrid = T_inf*ones(n, m);
objGrid = false(n, m);
%
```

```

% create cylinder
radius = 25;
cyl_cen_x = 100;
cyl_cen_y =
ceil(m/2);
% for i = 1 : n    for j = 1 : m    if (sqrt((i -
cyl_cen_x)^2 + (j - cyl_cen_y)^2) < radius)
objGrid(i, j) = true;    end
    end
end
% set boundary conditions for j = 1 : m % left and
right boundaries    psiGrid(1, j) = (j - ceil(m/2))
* delta_h * U_inf;    psiGrid(n, j) = (j -
ceil(m/2)) * delta_h * U_inf;
end
% top and bottom boundaries psiGrid(:, 1) = (1 -
ceil(m/2)) * delta_h * U_inf; psiGrid(:, m) = (m
- ceil(m/2)) * delta_h * U_inf;
% temp boundaries for
j = 2 : m - 1    for i
= 2 : n - 1    if
objGrid(i, j)
tempGrid(i, j) = 400;
end    end end
%% model changes begin for
time = 1 : time_intervals
    % update u and v values for omega
calculation    for j = 2 : m - 1    for i
= 2 : n - 1
        uGrid(i, j) = (psiGrid(i, j+1) - psiGrid(i, j-1))/(2*delta_h);
vGrid(i, j) = (psiGrid(i-1, j) - psiGrid(i+1, j))/(2*delta_h);    end
    end
    % iterate through omega grid until errors are below threshold
omegaGrid = iterate_omegagrid(psiGrid, objGrid, omegaGrid, uGrid, vGrid,
delta_h, delta_t, kin_viscosity, n, m);
    % iterate through omega grid until errors are below threshold
tempGrid = iterate_tempgrid(objGrid, tempGrid, uGrid, vGrid, delta_h,
delta_t, therm_diffusivity, n, m);
    % iterate through psi grid until errors are below
threshold    max_error = Inf;
    while max_error > threshold_error
        [psiGrid, max_error] = iterate_psigrid(psiGrid, objGrid,
omegaGrid, delta_h, n, m, F_overrelax);    end
    % store every 1 in few frames to avoid overloading :)
if mod(time, frame_store_rate) == 1    numFrame =
numFrame + 1;    psicell{numFrame} = psiGrid;
omegacell{numFrame} = omegaGrid;    tempcell{numFrame} =
tempGrid;    disp("Created frame " + string(numFrame) + "
at time " + string(datetime('now')));    end end
%% create video videoFinal =
VideoWriter('CylinderVideoMAE423.mp4', 'MPEG-4');
open(videoFinal);

```

```

% loop through frames and store
them for f = 1 : numFrame    psi =
psicell{f}';    omega =
flipud(omegacell{f}');    temp =
tempcell{f}';    subplot(3, 1, 1)
cla;    imagesc(psi);    hold on
contour(psi, 40, 'k');    hold off
    colormap(jet);
axis equal tight;
colorbar;
    title('Streamfunction');    drawnow;    subplot(3, 1, 2)
cla;    imagesc(omega)    colormap(jet);    axis equal
tight;    ax = gca; % Get current axes handle    ax.CLim =
[-0.05, 0.05]; % Set color limits    colorbar;
title('Vorticity');    drawnow;    subplot(3, 1, 3)    cla;
imagesc(temp)    colormap(jet);    axis equal tight;
colorbar;    title('Temperature');    drawnow;
sgtitle("Time step No. " + string(f*frame_store_rate - 4));
frame = getframe(gcf); % get current frame
writeVideo(videoFinal, frame); % add frame to video end
close(videoFinal);
%% iterate through omega grid function newGrid = iterate_omegagrid(psiGrid,
objGrid, oldOmegaGrid, u, v, h, t, nu, n, m)    modOmegaGrid =
oldOmegaGrid;    for j = 2 : m - 1    for i = 2 : n - 1    if
objGrid(i, j)    modOmegaGrid(i, j) = -2*(psiGrid(i+1, j) +
psiGrid(i-1, j) + psiGrid(i, j+1) + psiGrid(i, j-1) - 4*psiGrid(i,
j))/(h^2);    end    end    end    for j = 2 : m - 1    for
i = 2 : n - 1
        if ~objGrid(i, j)    if u(i, j) < 0
delta_u_omega = u(i+1, j)*modOmegaGrid(i+1, j) - u(i, j)*modOmegaGrid(i,
j);    else    delta_u_omega = u(i,
j)*modOmegaGrid(i, j) - u(i-1,
j)*modOmegaGrid(i-1, j);
        end
    if v(i, j) < 0
        delta_v_omega = v(i, j+1)*modOmegaGrid(i, j+1) -
v(i, j)*modOmegaGrid(i, j);    else
        delta_v_omega = v(i, j)*modOmegaGrid(i, j) -
v(i, j-1)*modOmegaGrid(i, j-1);    end
        laplacian_omega = (modOmegaGrid(i+1, j) + modOmegaGrid(i-1, j)
+ modOmegaGrid(i, j-1) + modOmegaGrid(i, j+1) - 4*modOmegaGrid(i, j))/(h^2);
modOmegaGrid(i, j) = modOmegaGrid(i, j) + t*(-1*(delta_u_omega)/h -
(delta_v_omega)/h + nu*laplacian_omega);    end    end    end
    modOmegaGrid(n, :) = modOmegaGrid(n-1,
:);    newGrid = modOmegaGrid; end
%% iterate through temp grid function newGrid = iterate_tempgrid(objGrid,
oldTempGrid, u, v, h, t, alpha, n, m)
    modTempGrid = oldTempGrid;    for j = 2 : m - 1    for i =
2 : n - 1
        if ~objGrid(i, j)    if u(i, j) < 0
delta_u_temp = u(i, j)*modTempGrid(i+1, j) - u(i,
j)*modTempGrid(i, j);    else
delta_u_temp = u(i, j)*modTempGrid(i, j) - u(i,

```

```

j)*modTempGrid(i-1, j);                                end                                if v(i,
j) < 0                                delta_v_temp = v(i, j)*modTempGrid(i,
j+1) - v(i, j)*modTempGrid(i, j);                                else
delta_v_temp = v(i, j)*modTempGrid(i, j) - v(i,
j)*modTempGrid(i, j-1);                                end                                laplacian_temp =
(modTempGrid(i+1, j) + modTempGrid(i-1, j) + modTempGrid(i, j-1) +
modTempGrid(i, j+1) - 4*modTempGrid(i, j))/(h^2);
modTempGrid(i, j) = modTempGrid(i, j) + t*(-1*(delta_u_temp)/h -
(delta_v_temp)/h + alpha*laplacian_temp);
end                                end                                end                                modTempGrid(n,
:) = modTempGrid(n-1, :);                                newGrid =
modTempGrid; end
%% iterate through psi grid using over-relaxed finite-difference (Gauss-
Seidel) and also error
function [newGrid, max_error] = iterate_psigrid(oldPsiGrid, objGrid,
omegaGrid, h, n, m, F)                                modPsiGrid = oldPsiGrid;                                errorGrid = zeros(n,
m);                                for j = 2 : m - 1                                for i = 2 : n - 1
                                if objGrid(i, j)                                continue;                                end
                                % determine psi
                                modPsiGrid(i, j) = modPsiGrid(i, j) + F*0.25*(modPsiGrid(i-1, j) +
modPsiGrid(i+1, j) + modPsiGrid(i, j-1) + modPsiGrid(i, j+1) +
4*h*h*omegaGrid(i, j) - 4*modPsiGrid(i, j));
                                % determine error                                errorGrid(i, j) =
abs(modPsiGrid(i, j) - oldPsiGrid(i, j)) / oldPsiGrid(i, j);
end                                modPsiGrid(n, j) = 2*modPsiGrid(n-1, j) - modPsiGrid(n-2,
j);                                end
                                newGrid = modPsiGrid;                                max_error =
max(errorGrid, [], 'all'); end

```

2) Plane with rectangular wings:

```

% MAE 423 Final Project
%
% By Dini Kathriarachchi & Allison Lesser
% set constants for problem n = 500; % vertical
length m = 200; % horizontal length
threshold_error = 0.05; F_overrelax = 1.5;
time_intervals = 10000; numFrame = 0;
frame_store_rate = 10; % how many frames are
stored
% constants for air
U_inf = 5; % m/s T_inf = 300; % K therm_diffusivity = 2.2
* 10^-5; % m^2/s kin_viscosity = 1.5 * 10^-5; Re = 200; %
no units delta_h = Re * kin_viscosity / (50*U_inf); %
delta_t = delta_h/(10*U_inf); % create cell to store
frames psicell =
cell(ceil(time_intervals/frame_store_rate), 1); omegacell
= cell(ceil(time_intervals/frame_store_rate), 1);
tempcell = cell(ceil(time_intervals/frame_store_rate),
1);
% create grids of desired geometry (n x
m) psiGrid = zeros(n, m); omegaGrid =

```

```

zeros(n, m); uGrid = zeros(n, m); vGrid =
zeros(n, m); tempGrid = T_inf*ones(n, m);
objGrid = false(n, m);
%
% rectangle wings wlength = 50; wwidth = 15; coord_x = 100; for i =
coord_x - ceil(wwidth/2) : coord_x + floor(wwidth/2)    for j =
ceil(m/2) - ceil(wlength/2) : ceil(m/2) + floor(wlength/2)
objGrid(i, j) = true;    end end % plane body plength = 15; pwidth
= 75; coord_x = 110; for i = 1 : n    for j = 1 : m
    if ((j - ceil(m/2))^2/(plength/2)^2 + (i - coord_x)^2/(pwidth/2)^2 <=
1)        objGrid(i, j) = true;        end    end end
% set boundary conditions for j = 1 : m % left and
right boundaries    psiGrid(1, j) = (j - ceil(m/2))
* delta_h * U_inf;    psiGrid(n, j) = (j -
ceil(m/2)) * delta_h * U_inf; end
% top and bottom boundaries psiGrid(:, 1) = (1 -
ceil(m/2)) * delta_h * U_inf; psiGrid(:, m) = (m
- ceil(m/2)) * delta_h * U_inf;
% temp boundaries for j = 2 : m
- 1    for i = 2 : n - 1
if objGrid(i, j)
tempGrid(i, j) = 400;
end    end end
%% model changes begin for
time = 1 : time_intervals
    % update u and v values for omega calculation
    for j = 2 : m - 1
        for i = 2 : n - 1
            uGrid(i, j) = (psiGrid(i, j+1) - psiGrid(i, j-1))/(2*delta_h);
vGrid(i, j) = (psiGrid(i-1, j) - psiGrid(i+1, j))/(2*delta_h);
        end    end
        % iterate through omega grid until errors are below threshold
omegaGrid = iterate_omegagrid(psiGrid, objGrid, omegaGrid, uGrid, vGrid,
delta_h, delta_t, kin_viscosity, n, m);
        % iterate through omega grid until errors are below threshold
tempGrid = iterate_tempgrid(objGrid, tempGrid, uGrid, vGrid, delta_h,
delta_t, therm_diffusivity, n, m);
        % iterate through psi grid until errors are below
threshold    max_error = Inf;    while max_error >
threshold_error
        [psiGrid, max_error] = iterate_psigrid(psiGrid, objGrid,
omegaGrid, delta_h, n, m, F_overrelax);    end
        % store every 1 in few frames to avoid overloading :)
if mod(time, frame_store_rate) == 1        numFrame =
numFrame + 1;        psicell{numFrame} = psiGrid;
omegacell{numFrame} = omegaGrid;        tempcell{numFrame} =
tempGrid;        disp("Created frame " + string(numFrame) + "
at time " + string(datetime('now')));    end end
%% create video videoFinal =
VideoWriter('RectangleVideoMAE423.mp4', 'MPEG-4');
open(videoFinal);
% loop through frames and store
them for f = 1 : numFrame    psi =

```

```

psicell{f}';    omega =
flipud(omegacell{f}');    temp =
tempcell{f}';    subplot(3, 1, 1)
cla;    imagesc(psi);    hold on
contour(psi, 40, 'k');    hold off
colormap(jet);    axis equal tight;
colorbar;
title('Streamfunction');
drawnow;    subplot(3, 1, 2)
cla;    imagesc(omega)
colormap(jet);
axis equal tight;
ax = gca; % Get current axes handle
ax.CLim = [-0.05, 0.05]; % Set color limits
colorbar;    title('Vorticity');    drawnow;
subplot(3, 1, 3)    cla;    images(temp)
colormap(jet);    axis equal tight;
colorbar;    title('Temperature');
drawnow;

sgtitle("Time step No. " + string(f*frame_store_rate -
4));    frame = getframe(gcf); % get current frame
writeVideo(videoFinal, frame); % add frame to video end
close(videoFinal);

%% iterate through omega grid
function newGrid = iterate_omegagrid(psiGrid, objGrid, oldOmegaGrid, u, v,
h, t, nu, n, m)    modOmegaGrid = oldOmegaGrid;    for j = 2 : m - 1
for i = 2 : n - 1    if objGrid(i, j)    modOmegaGrid(i,
j) = -2*(psiGrid(i+1, j) + psiGrid(i-1, j) + psiGrid(i, j+1) + psiGrid(i,
j-1) - 4*psiGrid(i, j))/(h^2);    end    end    end    for j = 2
: m - 1    for i = 2 : n - 1
if ~objGrid(i, j)    if u(i, j) < 0
delta_u_omega = u(i+1, j)*modOmegaGrid(i+1, j) - u(i, j)*modOmegaGrid(i,
j);    else    delta_u_omega = u(i,
j)*modOmegaGrid(i, j) - u(i-1,
j)*modOmegaGrid(i-1, j);    end    if v(i, j)
< 0    delta_v_omega = v(i, j+1)*modOmegaGrid(i, j+1)
- v(i, j)*modOmegaGrid(i, j);    else
delta_v_omega = v(i, j)*modOmegaGrid(i, j) - v(i, j-1)*modOmegaGrid(i,
j-1);    end
laplacian_omega = (modOmegaGrid(i+1, j) + modOmegaGrid(i-1, j)
+ modOmegaGrid(i, j-1) + modOmegaGrid(i, j+1) - 4*modOmegaGrid(i, j))/(h^2);
modOmegaGrid(i, j) = modOmegaGrid(i, j) + t*(-1*(delta_u_omega)/h -
(delta_v_omega)/h + nu*laplacian_omega);    end    end    end
modOmegaGrid(n, :) = modOmegaGrid(n-1, :);    newGrid = modOmegaGrid; end

%% iterate through temp grid
function newGrid = iterate_tempgrid(objGrid, oldTempGrid, u, v, h, t, alpha,
n, m)    modTempGrid = oldTempGrid;    for j = 2 : m - 1    for i = 2 : n
- 1
if ~objGrid(i, j)    if u(i, j) < 0
delta_u_temp = u(i, j)*modTempGrid(i+1, j) -
u(i, j)*modTempGrid(i, j);    else
delta_u_temp = u(i, j)*modTempGrid(i, j) - u(i,

```

```

j)*modTempGrid(i-1, j);                                end                                if v(i,
j) < 0                                delta_v_temp = v(i, j)*modTempGrid(i,
j+1) - v(i, j)*modTempGrid(i, j);                                else
delta_v_temp = v(i, j)*modTempGrid(i, j) - v(i,
j)*modTempGrid(i, j-1);                                end                                laplacian_temp =
(modTempGrid(i+1, j) + modTempGrid(i-1, j) + modTempGrid(i, j-1) +
modTempGrid(i, j+1) - 4*modTempGrid(i, j))/(h^2);
modTempGrid(i, j) = modTempGrid(i, j) + t*(-1*(delta_u_temp)/h -
(delta_v_temp)/h + alpha*laplacian_temp);
end                                end                                end                                modTempGrid(n,
:) = modTempGrid(n-1, :);                                newGrid =
modTempGrid; end
%% iterate through psi grid using over-relaxed finite-difference (Gauss-
Seidel) and also error function [newGrid, max_error] =
iterate_psigrid(oldPsiGrid, objGrid, omegaGrid, h, n, m, F)                                modPsiGrid =
oldPsiGrid;                                errorGrid = zeros(n, m);                                for j = 2 : m - 1                                for i = 2
: n - 1
                                if objGrid(i, j)
continue;                                end
                                % determine psi                                modPsiGrid(i, j) = modPsiGrid(i, j) +
F*0.25*(modPsiGrid(i-1, j) + modPsiGrid(i+1, j) + modPsiGrid(i, j-1) +
modPsiGrid(i, j+1) +
4*h*h*omegaGrid(i, j) - 4*modPsiGrid(i, j));
                                % determine error
                                errorGrid(i, j) = abs(modPsiGrid(i, j) - oldPsiGrid(i, j))
/ oldPsiGrid(i, j);                                end
                                modPsiGrid(n, j) = 2*modPsiGrid(n-1, j) - modPsiGrid(n-2,
j);                                end                                newGrid = modPsiGrid;                                max_error = max(errorGrid,
[], 'all'); end

```

3) Plane with elliptical wings:

```

% MAE 423 Final Project
%
% By Dini Kathriarachchi & Allison Lesser
%
% set constants for problem n = 500; % vertical
length m = 200; % horizontal length
threshold_error = 0.05; F_overrelax = 1.5;
time_intervals = 10000; numFrame = 0;
frame_store_rate = 10; % how many frames are
stored
% constants for air
U_inf = 5; % m/s T_inf = 300; % K therm_diffusivity = 2.2
* 10^-5; % m^2/s kin_viscosity = 1.5 * 10^-5; Re = 200; %
no units delta_h = Re * kin_viscosity / (50*U_inf); %
delta_t = delta_h/(10*U_inf); % create cell to store
frames psicell =
cell(ceil(time_intervals/frame_store_rate), 1); omegacell
= cell(ceil(time_intervals/frame_store_rate), 1);
tempcell = cell(ceil(time_intervals/frame_store_rate),
1);

```

```

% create grids of desired geometry (n x
m) psiGrid = zeros(n, m); omegaGrid =
zeros(n, m); uGrid = zeros(n, m); vGrid =
zeros(n, m); tempGrid = T_inf*ones(n, m);
objGrid = false(n, m);
%
% ellipsoid
wings wlength =
50; wwidth = 15;
coord_x = 100;
for i = 1 : n
for j = 1 : m
    if ((j - ceil(m/2))^2/(wlength/2)^2 + (i - coord_x)^2/(wwidth/2)^2 <=
1)
        objGrid(i, j) = true;
    end
end end % plane body
plength = 15; pwidth = 75; coord_x = 110;
for i = 1 : n
    for j = 1 : m
        if ((j - ceil(m/2))^2/(plength/2)^2 + (i - coord_x)^2/(pwidth/2)^2 <= 1)
            objGrid(i, j) = true;
        end
    end
end
% set boundary conditions for j = 1 : m % left and
right boundaries
psiGrid(1, j) = (j - ceil(m/2))
* delta_h * U_inf;
psiGrid(n, j) = (j -
ceil(m/2)) * delta_h * U_inf;
end
% top and bottom boundaries
psiGrid(:, 1) = (1 -
ceil(m/2)) * delta_h * U_inf;
psiGrid(:, m) = (m
- ceil(m/2)) * delta_h * U_inf;
% temp boundaries
for j = 2 : m
- 1
    for i = 2 : n - 1
if objGrid(i, j)
tempGrid(i, j) = 400;
end
end end
%% model changes begin for
time = 1 : time_intervals
    % update u and v values for omega calculation
    for j = 2 : m - 1
        for i = 2 : n - 1
            uGrid(i, j) = (psiGrid(i, j+1) - psiGrid(i, j-
1))/(2*delta_h);
            vGrid(i, j) = (psiGrid(i-1, j) -
psiGrid(i+1, j))/(2*delta_h);
        end
    end
    % iterate through omega grid until errors are below threshold
omegaGrid = iterate_omegagrid(psiGrid, objGrid, omegaGrid, uGrid, vGrid,
delta_h, delta_t, kin_viscosity, n, m);
    % iterate through omega grid until errors are below threshold
tempGrid = iterate_tempgrid(objGrid, tempGrid, uGrid, vGrid, delta_h,
delta_t, therm_diffusivity, n, m);
    % iterate through psi grid until errors are below
threshold
    max_error = Inf;
    while max_error > threshold_error
        [psiGrid, max_error] = iterate_psigrid(psiGrid, objGrid,
omegaGrid, delta_h, n, m, F_overrelax);
    end
    % store every 1 in few frames to avoid overloading
    :)
    if mod(time, frame_store_rate) == 1
numFrame = numFrame + 1;
        psicell{numFrame} =
psiGrid;
        omegacell{numFrame} = omegaGrid;
        tempcell{numFrame} = tempGrid;
    end
end

```



```

        disp("Created frame " + string(numFrame) + " at time "
+ string(datetime('now')));    end end
%
%% create video
videoFinal = VideoWriter('EllipsoidVideoMAE423.mp4', 'MPEG-4');
open(videoFinal);
% loop through frames and store them for f =
1 : numFrame    psi = psicell{f}';    omega
= flipud(omegacell{f}');    temp =
tempcell{f}';    subplot(3, 1, 1)    cla;
imagesc(psi);    hold on    contour(psi, 40,
'k');    hold off    colormap(jet);    axis
equal tight;    colorbar;
title('Streamfunction');    drawnow;
subplot(3, 1, 2)    cla;    imagesc(omega)
colormap(jet);    axis equal tight;    ax =
gca; % Get current axes handle    ax.CLim =
[-0.05, 0.05]; % Set color limits
colorbar;    title('Vorticity');    drawnow;
subplot(3, 1, 3)    cla;    images(temp)
colormap(jet);    axis equal tight;
colorbar;    title('Temperature');
drawnow;

sgtitle("Time step No. " + string(f*frame_store_rate -
4));    frame = getframe(gcf); % get current frame
writeVideo(videoFinal, frame); % add frame to video end
close(videoFinal);
%% iterate through omega grid function newGrid = iterate_omegagrid(psiGrid,
objGrid, oldOmegaGrid, u, v, h, t, nu, n, m)    modOmegaGrid =
oldOmegaGrid;    for j = 2 : m - 1    for i = 2 : n - 1    if
objGrid(i, j)    modOmegaGrid(i, j) = -2*(psiGrid(i+1, j) +
psiGrid(i-1, j) + psiGrid(i, j+1) + psiGrid(i, j-1) - 4*psiGrid(i,
j))/(h^2);    end    end    end    for j = 2 : m - 1    for
i = 2 : n - 1
        if ~objGrid(i, j)    if u(i, j) < 0
delta_u_omega = u(i+1, j)*modOmegaGrid(i+1, j) - u(i, j)*modOmegaGrid(i,
j);    else    delta_u_omega = u(i,
j)*modOmegaGrid(i, j) - u(i-1,
j)*modOmegaGrid(i-1, j);    end    if v(i, j) < 0
delta_v_omega = v(i, j+1)*modOmegaGrid(i, j+1) - v(i, j)*modOmegaGrid(i, j);
else    delta_v_omega = v(i, j)*modOmegaGrid(i, j) - v(i, j-
1)*modOmegaGrid(i, j-1);    end    laplacian_omega =
(modOmegaGrid(i+1, j) + modOmegaGrid(i-1, j) + modOmegaGrid(i, j-1) +
modOmegaGrid(i, j+1) - 4*modOmegaGrid(i, j))/(h^2);
modOmegaGrid(i, j) = modOmegaGrid(i, j) + t*(-1*(delta_u_omega)/h -
(delta_v_omega)/h + nu*laplacian_omega);

end
end    end
    modOmegaGrid(n, :) = modOmegaGrid(n-1,
:);    newGrid = modOmegaGrid; end
%% iterate through temp grid function newGrid = iterate_tempgrid(objGrid,
oldTempGrid, u, v, h, t, alpha, n, m)

```

```

    modTempGrid = oldTempGrid;    for j = 2 : m - 1        for i =
2 : n - 1
    if ~objGrid(i, j)                if u(i, j) < 0
delta_u_temp = u(i, j)*modTempGrid(i+1, j) - u(i,
j)*modTempGrid(i, j);                else
delta_u_temp = u(i, j)*modTempGrid(i, j) - u(i,
j)*modTempGrid(i-1, j);
end                                if v(i,
j) < 0
                                delta_v_temp = v(i, j)*modTempGrid(i, j+1) -
v(i, j)*modTempGrid(i, j);                else
                                delta_v_temp = v(i, j)*modTempGrid(i, j) - v(i,
j)*modTempGrid(i, j-1);                end                                laplacian_temp =
(modTempGrid(i+1, j) + modTempGrid(i-1, j) + modTempGrid(i, j-1) +
modTempGrid(i, j+1) - 4*modTempGrid(i, j))/(h^2);
modTempGrid(i, j) = modTempGrid(i, j) + t*(-1*(delta_u_temp)/h -
(delta_v_temp)/h + alpha*laplacian_temp);
end                                end                                end                                modTempGrid(n,
:) = modTempGrid(n-1, :);    newGrid =
modTempGrid; end
%% iterate through psi grid using over-relaxed finite-difference (Gauss-
Seidel) and also error function [newGrid, max_error] =
iterate_psigrid(oldPsiGrid, objGrid, omegaGrid, h, n, m, F)    modPsiGrid =
oldPsiGrid;    errorGrid = zeros(n, m);    for j = 2 : m - 1        for i = 2
: n - 1
                                if objGrid(i, j)                continue;                end
                                % determine psi
                                modPsiGrid(i, j) = modPsiGrid(i, j) + F*0.25*(modPsiGrid(i-1, j) +
modPsiGrid(i+1, j) + modPsiGrid(i, j-1) + modPsiGrid(i, j+1) +
4*h*h*omegaGrid(i, j) - 4*modPsiGrid(i, j));
                                % determine error                                errorGrid(i, j) =
abs(modPsiGrid(i, j) - oldPsiGrid(i, j)) / oldPsiGrid(i, j);
end                                modPsiGrid(n, j) = 2*modPsiGrid(n-1, j) - modPsiGrid(n-2,
j);    end
    newGrid = modPsiGrid;    max_error =
max(errorGrid, [], 'all'); end

```

4) Plane with tapered wings:

```

% MAE 423 Final Project
%
% By Dini Kathriarachchi & Allison Lesser
%
% set constants for problem n = 500; % vertical
length m = 200; % horizontal length
threshold_error = 0.05; F_overrelax = 1.5;
time_intervals = 10000; numFrame = 0;
frame_store_rate = 10; % how many frames are
stored
% constants for air
U_inf = 5; % m/s T_inf = 300; % K therm_diffusivity = 2.2
* 10^-5; % m^2/s kin_viscosity = 1.5 * 10^-5; Re = 200; %

```

```

no units delta_h = Re * kin_viscosity / (50*U_inf); %
delta_t = delta_h/(10*U_inf); % create cell to store
frames psicell =
cell(ceil(time_intervals/frame_store_rate), 1); omegacell
= cell(ceil(time_intervals/frame_store_rate), 1);
tempcell = cell(ceil(time_intervals/frame_store_rate),
1);
% create grids of desired geometry (n x
m) psiGrid = zeros(n, m); omegaGrid =
zeros(n, m); uGrid = zeros(n, m); vGrid =
zeros(n, m); tempGrid = T_inf*ones(n, m);
objGrid = false(n, m);
%
% tapered wings
base_height = 10; wlength
= 50; coord_x = 95;
for i = coord_x - int32(2/3*base_height) : coord_x +
int32(1/3*base_height)      y = i - coord_x + int32(2/3*base_height);
for j = 1 : m
    if (abs(j - ceil(m/2)) <= y*0.5*wlength/base_height)
objGrid(i, j) = true;          end      end end wlength = 50; wwidth =
10; coord_x = 100; for i = coord_x - ceil(wwidth/3) : coord_x +
floor(2*wwidth/3)      for j = ceil(m/2) - ceil(wlength/2) :
ceil(m/2) + floor(wlength/2)      objGrid(i, j) = true;      end
end % plane body plength = 15; pwidth = 75; coord_x = 110; for i =
1 : n      for j = 1 : m
        if ((j - ceil(m/2))^2/(plength/2)^2 + (i - coord_x)^2/(pwidth/2)^2 <=
1)      objGrid(i, j) = true;          end      end end
% set boundary conditions for j = 1 : m % left and
right boundaries      psiGrid(1, j) = (j - ceil(m/2))
* delta_h * U_inf;      psiGrid(n, j) = (j -
ceil(m/2)) * delta_h * U_inf; end
% top and bottom boundaries psiGrid(:, 1) = (1 -
ceil(m/2)) * delta_h * U_inf; psiGrid(:, m) = (m
- ceil(m/2)) * delta_h * U_inf;
% temp boundaries for j = 2 : m
- 1      for i = 2 : n - 1
if objGrid(i, j)
tempGrid(i, j) = 400;
end      end end
%% model changes begin for
time = 1 : time_intervals
    % update u and v values for omega calculation
    for j = 2 : m - 1
        for i = 2 : n - 1
            uGrid(i, j) = (psiGrid(i, j+1) - psiGrid(i, j-1))/(2*delta_h);
vGrid(i, j) = (psiGrid(i-1, j) - psiGrid(i+1, j))/(2*delta_h);      end
        end
        % iterate through omega grid until errors are below threshold
omegaGrid = iterate_omegagrid(psiGrid, objGrid, omegaGrid, uGrid, vGrid,
delta_h, delta_t, kin_viscosity, n, m);

```

```

    % iterate through omega grid until errors are below threshold
tempGrid = iterate_tempgrid(objGrid, tempGrid, uGrid, vGrid, delta_h,
delta_t, therm_diffusivity, n, m);
    % iterate through psi grid until errors are below
threshold    max_error = Inf;    while max_error >
threshold_error
        [psiGrid, max_error] = iterate_psigrid(psiGrid, objGrid,
omegaGrid, delta_h, n, m, F_overrelax);    end
    % store every 1 in few frames to avoid overloading :)
if mod(time, frame_store_rate) == 1    numFrame =
numFrame + 1;    psicell{numFrame} = psiGrid;
omegacell{numFrame} = omegaGrid;    tempcell{numFrame} =
tempGrid;    disp("Created frame " + string(numFrame) + "
at time " + string(datetime('now')));    end end
%
%% create video videoFinal =
VideoWriter('TaperedVideoMAE423.mp4', 'MPEG-4');
open(videoFinal);
% loop through frames and store
them for f = 1 : numFrame    psi =
psicell{f}';    omega =
flipud(omegacell{f}');    temp =
tempcell{f}';    subplot(3, 1, 1)
cla;    imagesc(psi);    hold on
contour(psi, 40, 'k');    hold off
colormap(jet);    axis equal tight;
colorbar;
title('Streamfunction');
drawnow;    subplot(3, 1, 2)
cla;
    imagesc(omega)
colormap(jet);
axis equal tight;
    ax = gca; % Get current axes handle    ax.CLim = [-0.05,
0.05]; % Set color limits    colorbar;
title('Vorticity');    drawnow;    subplot(3, 1, 3)    cla;
imagesc(temp)    colormap(jet);    axis equal tight;
colorbar;    title('Temperature');    drawnow;
sgtitle("Time step No. " + string(f*frame_store_rate - 4));
frame = getframe(gcf); % get current frame
writeVideo(videoFinal, frame); % add frame to video end
close(videoFinal);
%% iterate through omega grid function newGrid = iterate_omegagrid(psiGrid,
objGrid, oldOmegaGrid, u, v, h, t, nu, n, m)
    modOmegaGrid = oldOmegaGrid;    for j = 2 : m - 1    for i = 2 : n
- 1
        if objGrid(i, j)    modOmegaGrid(i, j) = -
2*(psiGrid(i+1, j) + psiGrid(i-1, j) + psiGrid(i, j+1) + psiGrid(i, j-1)
- 4*psiGrid(i, j))/(h^2);    end    end    end    for j = 2 :
m - 1
        for i = 2 : n - 1
            if ~objGrid(i, j)    if u(i, j) < 0
delta_u_omega = u(i+1, j)*modOmegaGrid(i+1, j) - u(i, j)*modOmegaGrid(i,
j);
            else
delta_u_omega = u(i,
j)*modOmegaGrid(i, j) - u(i-1,

```

```

j)*modOmegaGrid(i-1, j);                                end                                if v(i, j)
< 0                                delta_v_omega = v(i, j+1)*modOmegaGrid(i, j+1)
- v(i, j)*modOmegaGrid(i, j);                                else
delta_v_omega = v(i, j)*modOmegaGrid(i, j) - v(i, j-1)*modOmegaGrid(i,
j-1);
                                end                                laplacian_omega = (modOmegaGrid(i+1, j) +
modOmegaGrid(i-1, j) + modOmegaGrid(i, j-1) + modOmegaGrid(i, j+1) -
4*modOmegaGrid(i, j))/(h^2);                                modOmegaGrid(i, j) =
modOmegaGrid(i, j) + t*(-1*(delta_u_omega)/h - (delta_v_omega)/h +
nu*laplacian_omega);                                end                                end                                end
modOmegaGrid(n, :) = modOmegaGrid(n-1,
:);    newGrid = modOmegaGrid; end
%% iterate through temp grid
function newGrid = iterate_tempgrid(objGrid, oldTempGrid, u, v, h, t, alpha,
n, m)    modTempGrid = oldTempGrid;    for j = 2 : m - 1    for i = 2 : n
- 1
        if ~objGrid(i, j)                                if u(i, j) < 0
delta_u_temp = u(i, j)*modTempGrid(i+1, j) - u(i, j)*modTempGrid(i, j);
else                                delta_u_temp = u(i, j)*modTempGrid(i, j) - u(i,
j)*modTempGrid(i-1, j);                                end                                if v(i,
j) < 0                                delta_v_temp = v(i, j)*modTempGrid(i,
j+1) - v(i, j)*modTempGrid(i, j);                                else
delta_v_temp = v(i, j)*modTempGrid(i, j) - v(i,
j)*modTempGrid(i, j-1);                                end                                laplacian_temp =
(modTempGrid(i+1, j) + modTempGrid(i-1, j) + modTempGrid(i, j-1) +
modTempGrid(i, j+1) - 4*modTempGrid(i, j))/(h^2);
modTempGrid(i, j) = modTempGrid(i, j) + t*(-1*(delta_u_temp)/h -
(delta_v_temp)/h + alpha*laplacian_temp);
end                                end                                end    modTempGrid(n,
:) = modTempGrid(n-1, :);    newGrid =
modTempGrid; end
%% iterate through psi grid using over-relaxed finite-difference (Gauss-
Seidel) and also error function [newGrid, max_error] =
iterate_psigrid(oldPsiGrid, objGrid, omegaGrid, h, n, m, F)    modPsiGrid =
oldPsiGrid;    errorGrid = zeros(n, m);    for j = 2 : m - 1
        for i = 2 : n - 1
            if objGrid(i,
j)
continue;                                end
                % determine psi
                modPsiGrid(i, j) = modPsiGrid(i, j) + F*0.25*(modPsiGrid(i-1, j) +
modPsiGrid(i+1, j) + modPsiGrid(i, j-1) + modPsiGrid(i, j+1) +
4*h*h*omegaGrid(i, j) - 4*modPsiGrid(i, j));
                % determine error
                errorGrid(i, j) = abs(modPsiGrid(i, j) - oldPsiGrid(i, j))
/ oldPsiGrid(i, j);                                end
                modPsiGrid(n, j) = 2*modPsiGrid(n-1, j) - modPsiGrid(n-2,
j);    end    newGrid = modPsiGrid;
max_error = max(errorGrid, [], 'all');
end

```

5) Nusselt number calculations:

```

n
= 500; % vertical length m =
200; % horizontal length
numFrame = 1000; c_air = 1005;
% J/kgK density_air = 1.17; %
kg/m3 k_air = 0.024; % W/mK
boundary_right = 500;
boundary_left = 1;
Q_over_A = zeros(numFrame, m);
% constants for air U_inf = 5; % m/s kin_viscosity = 1.5 * 10^-5; Re
= 200; % no units delta_h = Re * kin_viscosity / (50*U_inf); %
delta_t = delta_h/(10*U_inf); L = delta_h * 50; for f = 2 : numFrame
% go through all frames psi = taper_psi{f}; % get stored psi
values for each frame temp = taper_temp{f}; % get stored temp
values for each frame for j = 2 : m-1 % bottom to top
u_right = (psi(boundary_right, j+1) - psi(boundary_right, j-
1))/(2*delta_h); % calculate velocity at right boundary at each point
u_left = (psi(boundary_left, j+1) - psi(boundary_left,
j-1))/(2*delta_h); % calculate velocity at left boundary (should be = u_inf)
T_right = temp(boundary_right, j);
T_left = temp(boundary_left, j);
Q_over_A(f, j) = density_air * c_air * (T_right*u_right -
T_left*u_left);
% kg/m3 * J/kgK * K*m/s = J/(s*m2) = W / m2 = Q/A
end
end avg_heat_transfer = mean(Q_over_A, 'all'); % averaging across all frames
and all vertical grid cells
h_convect = avg_heat_transfer / (400 - 300);
Nusselt = h_convect * L / k_air;

```